Laboratorio de Inducción: Primeros Pasos con TurtleBot

Gabriel Sepúlveda V.

Octubre 2026

1 Conociendo al TurtleBot

La Figura 1 muestra una vista general del robot TurtleBot 2 [1], que utilizarán dentro de sus experiencias de laboratorio. Sus principales componentes son:

- TurtleBot: base robótica que contiene los elementos que permiten el movimiento del robot. Dentro de los elementos más relevantes, se encuentran:
  - 2 ruedas motorizadas en configuración diferencial
  - 1 tercera rueda de apoyo
  - 2 motores
  - 2 encoders
  - 1 controlador de motores
  - batería
  - panel de control
- Torre de dispositivos: Torre para la instalación de sensores, dispositivos de cómputo e interfaces con el usuario.
- Kinect: Sensor visual que provee imágenes RGBD para el sistema de percepción robótica.
- Parlante: Parlante para proporcionar información de estado a usuarios.
- Netbook: Dispositivo de cómputo para recibir información de sensores y controlar acciones del robot. En este computador deberán ejecutar sus aplicaciones en ROS2.

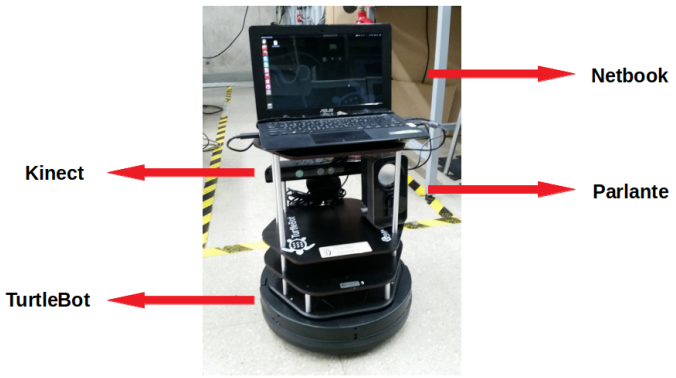


Figure 1: Vista general de TurtleBot 2 y sus principales dispositivos.

Como medida de seguridad, el TurtleBot tiene en la parte superior de su torre una correa de velcro que permite asegurar el netbook, tal como se aprecia en la Figura 2. Es sumamente importante que antes de

comenzar a ejecutar sus aplicaciones en ROS2, se asegure de que **el velcro de sujeción está sosteniendo el netbook de forma correcta**, porque de lo contrario, el computador podría caer desde la torre y sufrir daños.



Figure 2: Velcro de seguridad que permite asegurar netbook a la torre del TurtleBot.

En la Figura 3 se muestra la interfaz de energía de la base robótica. Ésta está compuesta por el plug de alimentación y el switch de encendido y apagado. Al llegar al laboratorio, usted encontrará el robot completamente cargado, y por lo tanto, no deberá interactuar con el plug de alimentación. Por otro lado, usted deberá utilizar el swtich de encendido y apagado para prender y apagar el robot cuando vaya a utilizarlo. Se recomienda mantener el robot apagado cuando no lo esté usando, para así aprovechar bien la carga de la batería.



Figure 3: Interfaz de energía del TurtleBot. Aquí se destaca el plug de conexión eléctrica que permite cargar la batería del robot y el switch de encendido y apagado.

Finalmente, la Figura 4 muestra el panel de control trasero del robot. Aquí se destacan los led de estado de la base robótica y el conector que comunica el robot con el computador. Este panel no deberá ser manipulado por usted, así es que en caso de que vea algo fuera de lo normal, consulte inmediatmente con el ayudante o profesor.

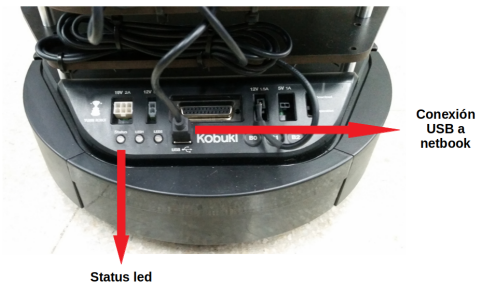


Figure 4: Panel de control trasero. Aquí encontrará los leds que indican el estado de la base robótica y el conector USB que permite la comunicación entre el netbook y el robot.

2 Preparación de Ambiente de Trabajo

Al llegar al laboratorio, cada grupo tendrá asignado un robot y un laptop. Dentro de cada laptop, usted encontrará una cuenta de usuario creada especialmente para su grupo. Esta cuenta de usuario pertenecerá exclusivamente a los integrantes de su grupo, y en ella pueden crear y almacenar todos los archivos y directorios que estimen conveniente.

Los nombres de usuario de cada cuenta siguen el patrón *grupo<num_de_grupo>*. Es decir, si usted pertenece al grupo X, para ingresar a su cuenta deberá utilizar los siguientes datos:

- Username: grupoX
- Password: grupoX

Para matener la privacidad de sus archivos, se recomienda que al ingresar por primera vez a sus cuentas modifiquen el password utilizando el comando *passwd*, tal como se muestra en la figura 5.

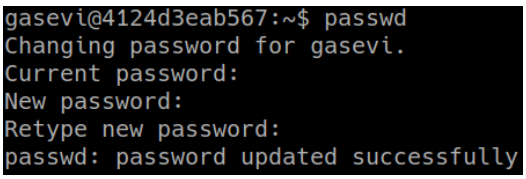


Figure 5: Cambio de password a través de la terminal.

Una vez dentro de su cuenta de usuario, verifique que el ambiente ROS se encuentra activo. Para ello, verifique que al final del archivo *.bashrc* se encuentren los comandos que permiten cargar el ambiente ROS y el workspace del Turtlebot. Si todo está correctamente configurado, usted debería visualizar las siguientes líneas al final del archivo *.bashrc*:

```
# ROS Environment

export ROS_DOMAIN_ID=X
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib/
source /opt/ros/humble/setup.bash
source /opt/kobuki_ws/install/local_setup.bash
```

Nota: Una vez que usted cree su workspace, deberá incluir la siguiente línea después de los comandos mostrados por el cuadro anterior:

```
source $HOME/<path_to_workspace>/install/local_setup.bash
```

3 Inicialización de Nodos de TurtleBot

Para poder utilizar el TurtleBot y sus sensores, usted deberá inicializar los nodos que controlan el hardware y permiten acceder a los recursos del robot (odometría, imagen de profundidad, imagen a color, comandos de velocidad, etc.). Al igual que con el simulador, dicha inicialización se realiza a partir de algunos archivos launch que vienen ya instalados en el sistema. A continuación se detallan los archivos launch que utilizaremos en nuestro laboratorio.

3.1 Kinect node

El nodo que controla los recursos del sensor *Kinect* se denomina *kinect_ros2_node*. Para levantarlo, abra una terminal y ejecute el siguiente comando:

```
ros2 run kinect_ros2 kinect_ros2_node
```

Para verificar que el nodo está funcionando de forma correcta, abra otra terminal e inicialice el nodo de visualización *rqt_image_view*, tal como se muestra a continuación

```
ros2 run rqt_image_view rqt_image_view
```

Luego de esto, usted debería ver aparecer una interfaz gráfica desde la cual puede seleccionar los tópicos publicados por Kinect y visualizar las imágenes de profundidad y color (RGB).

Si usted desea levantar el nodo que controla los recursos del *Kinect* directamente desde un archivo launch, debe incluir el tag que se muestra a continuación:

```
<node pkg="kinect_ros2" exec="kinect_ros2_node" name="kinect_ros2_node" namespace="kinect
    "/>
```

3.2 TurtleBot node

Nota: Antes de ejecutar los siguientes comandos, asegúrese de tener el robot posicionado en el piso, y nunca sobre los mesones.

El nodo que controla los recursos del *TurtleBot* se denomina *kobuki_node*. Para levantarlo, abra una terminal y ejecute el siguiente comando:

```
ros2 launch kobuki_node kobuki_node-launch.py
```

En otra terminal, envíe una velocidad rotacional constante de 1 [rad/s] utilizando el comando *ros2 topic pub*, tal como se muestra a continuación:

```
ros2 topic pub -r 5 /commands/velocity geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0,
    z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}"
```

Como resultado, usted debería ver al robot girar en sentido anti-horario.

Si usted desea levantar el nodo que controla los recursos del *TurtleBot* directamente desde un archivo launch, debe incluir el tag que se muestra a continuación:

```
<include file="$(find-pkg-share kobuki_node)/launch/kobuki_node-launch.py" />
```

4 Creando el ambiente de trabajo (*ROS Workspace*).

4.1 Creación e inicialización del *workspace*

Para establecer un *workspace* dentro de su cuenta de usuario, usted deberá crear una estructura de directorios destinada a contener todos los paquetes que desee crear en su proyecto, tal como se muestra a continuación:

```
mkdir -p $HOME/ros2_ws/src
```

Nota: El directorio base del workspace (*ros2_ws*) puede ser nombrado como usted desee, pero deberá ser consistente con el resto de los pasos.

4.2 Creación de paquetes

Para poder organizar y administrar sus códigos de forma ordenada, *ROS2* provee una estructura de desarrollo basada en paquetes. Para practicar la generación de paquetes, crearemos nuestro primer paquete de pruebas denominado *my_first_ros_pkg*, utilizando de los siguientes comandos:

```
cd $HOME/ros2_ws/src
ros2 pkg create my_first_ros_pkg --build-type=ament_cmake
```

4.3 Completando la estructura de directorios de nuestro paquete

En general, en este curso programaremos nuestros nodos utilizando el lenguaje de programación Python. Para poder organizar nuestros scripts y archivos lanzadores de forma ordenada, crearemos los siguientes subdirectorios:

- "nodes": subdirectorio donde residirán nuestros scripts
- "launch": subdirectorio donde residirán los lanzadores
- "my_first_ros_pkg": subdicrectorio que creará el módulo Python

A continuación se muestra la creación de los directorios anteriormente señalados:

```
cd $HOME/ros2_ws/src/my_first_ros_pkg
```

4.4 Nuestro primer nodo en *ROS2*: *rotate*

A continuación se presenta un script que hará que su robot gire a 1.0 [rad/s] en sentido anti-horario. Para ello, crearemos un nodo de nombre `RotateCmd` que publicará comandos de velocidad en el tópico `/commands/velocity`. A su vez, nuestro TurtleBot estará subscrito a dicho tópico para ejecutar los comandos de velocidad que sean recibidos.

Para lograr que nuestro robot gire de forma continua, el comando de velocidad deberá ser enviado de forma periódica con una frecuencia mayor a 2 [Hz]. Para lograr esta periodicidad, crearemos una callback que será llamada por un timer cada 0.1 [s]. Dentro de dicha callback, publicaremos el comando de velocidad utilizando una estructura de datos de tipo Twist.

A continuación se presenta el contenido del script *rotate.py*, el cual deberá ser creado dentro del directorio "$HOME/ros2_ws/src/my_first_ros_pkg/nodes":

```
#! /usr/bin/env python3

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Vector3, Twist

class RotateCmd( Node ):

  def __init__( self ):
    super().__init__( 'rotate_cmd' )
    self.MAX_BASE_ROT = 1.0 # [rad/s]
    self.rosaria_cmd_vel = self.create_publisher( Twist, '/commands/velocity', 1 )
    timer_period = 0.1  # seconds
    self.timer = self.create_timer(timer_period, self.timer_callback)

  def timer_callback(self):
    twist = Twist( linear = Vector3( x = 0.0, y = 0.0, z = 0.0 ),
                   angular = Vector3( x = 0.0, y = 0.0, z = self.MAX_BASE_ROT ) )
    self.rosaria_cmd_vel.publish( twist )

if __name__ == '__main__':
  rclpy.init()
  rotate_cmd = RotateCmd()
  rclpy.spin( rotate_cmd )
  rclpy.shutdown()
```

Nota: Una vez creado el archivo, asegúrese de que tenga permiso de ejecución. Si no es así, puede modificarlos con el comando: chmod u+x rotate.py

4.5 Agregar instrucciones para la construcción de paquete

Para indicarle a ROS donde se encuentran los scripts que conforman nuestro paquete, debemos modificar el archivo CMakeLists.txt, que se encuentra dentro del directorio: "$HOME/ros2_ws/src/my_first_ros_pkg/".

Dentro de él, debemos indicar:

- El nombre del paquete Python que deseamos instalar:

```
ament_python_install_package(${PROJECT_NAME})
```

- El nombre del o los scripts que conforman nuestro paquete

```
install(PROGRAMS nodes/rotate.py
        DESTINATION lib/${PROJECT_NAME})
```

- El nombre del dicretorio que contiene nuestros archivos lanzadores

```
install(DIRECTORY launch DESTINATION share/${PROJECT_NAME}/)
```

En resumen, nuestro arhcivo CMakeLists.txt deberá quedar como se muestra a continuación:

```
cmake_minimum_required(VERSION 3.8)
project(my_first_ros_pkg)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

# Install Python modules
ament_python_install_package(${PROJECT_NAME})

# Install Python executables
install(PROGRAMS nodes/rotate.py
        DESTINATION lib/${PROJECT_NAME})

# Install launch files
install(DIRECTORY launch DESTINATION share/${PROJECT_NAME}/)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

4.6 Compilación de paquetes pertenecientes al *workspace*

Para compilar el código fuente y activar los scrips generados en la estructura de directorios del workspace, ejecute el comando *colcon build –symlink-install* dentro del directorio base, tal como se muestra a continuación:

```
cd $HOME$/ros2_ws
colcon build --symlink-install
```

4.7 Incorporación del workspace al ambiente ROS

Para incorporar el workspace con los paquetes generados al ambiente ROS, se debe ejecutar el siguiente comando:

```
source $HOME/ros2_ws/install/local_setup.bash
```

Siguiendo el mismo procedimiento realizado para cargar de forma automática las variables de ambiente de ROS en nuestra cuenta de usuario, se puede automatizar la incroporación de nuestro workspace incluyendo el comando anterior al final del archivo $HOME/.bashrc.

4.8 Ejecución de scripts

Teniendo dos consolas *bash* independientes, ejecute el comando para levantar los nodos del TurtleBot y el nodo *rotate* que acabamos de crear, tal como se muestra a continuación:

```
console_1$ ros2 launch kobuki_node kobuki_node-launch.py
```

```
console_2$ ros2 run my_first_ros_pkg rotate.py
```

Pregunta: ¿Qué pasa si cambia el período de envío del comando de velocidad desde 0.1 [s] a 1 [s]?

5 Consideraciones prácticas

5.1 Filtrar imagen de profundidad

Tal como se comentó en clases, el Kinect está montado en la parte posterior del TurtleBot. Esto provoca que dentro del su campo de visión se alcance a ver uno de los pilares de su torre, tal como se muestra en la Figura 6.

Dado lo anterior, se recomienda determinar el rango de la imagen donde aparece dicho pilar y luego filtrarlo de los datos que van a ser utilizados para la detección de obstáculos.

5.2 Conversión de unidad de medida en imagen de profundidad

Como hemos visto a lo largo del semestre, la imagen de profundida generada por el simulador entrega mediciones en **metros**. La razón de utilizar esta convención es que la biblioteca OpenNI (utilizada tradicionalmente para acceder a los datos del Kinect) utiliza esa unidad de medida por defecto.

Sin embargo, el sensor Kinect ya no tiene soporte a través de OpenNI desde Ubuntu 20.04. Para resolver esto, ahora utilizaremos la biblioteca Freenect, la cual entrega la imagen de profundidad en **milímetros**. Dado esto, para hacer que sus códigos sean compatibles con el hardware, se recomienda tomar la imagen de profundidad en formato numpy y dividirla por 1000. De esta forma la imagen de profundidad será equivalente a la entregada por el simulador.

References

[1] https://www.turtlebot.com/turtlebot2/?_x_tr_sl&_x_tr_tl&_x_tr_hl

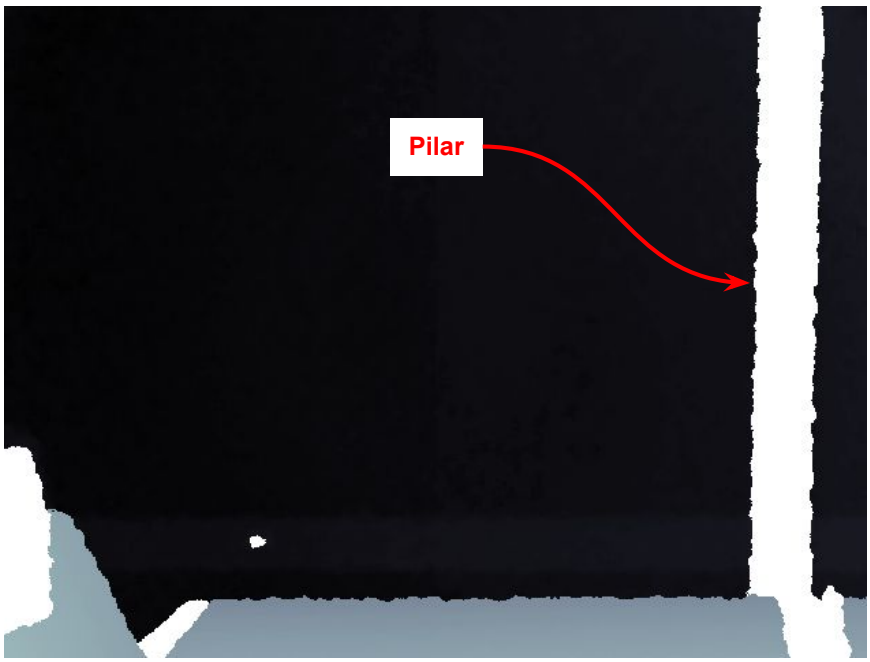


Figure 6: Imagen de profundidad tomada con Kinect montado en un TurtleBot.